SAHAYAK Presentation Script

Three speakers: Barakha, Shyamantika and Himanxhu

Project: SAHAYAK - A Real-Time Indian Sign Language Recognition System
Lead developer and project head: Johan Jyoti Kuli

How to use this script

This guide gives each speaker a role, speaking lines, what to show and a handover to the next speaker. Practise the meaning rather than memorising every word. Check the camera, backend and model before using the live website. Never claim an accuracy score unless a real test report supports it.

The project in one minute

SAHAYAK is a prototype that tries to recognize a limited set of trained signs and show their labels as text. It uses a browser camera, MediaPipe Hands, numeric hand features, a backend model and a website. Some extra features can translate text or speak it aloud.

The static two-hand path uses 126 values. The continuous path uses 132 values per frame across 10 time steps, making 1,320 values. The active labels can change, so check the live /api/status response before presenting. A label in a file alone does not prove the model recognizes it well.

Say "prototype" and "supported signs." Do not say "complete ISL translator."

Speaker 1 - Barakha Doley

Role: Introduction, Problem and Project Purpose

Suggested time: 1 minute 30 seconds

Say:

Good morning respected judges, teachers, and everyone here. We are presenting our project SAHAYAK, a real-time Indian Sign Language recognition prototype.

The word Sahayak means helper. We want to explore how a camera and a computer program could help show the name of a sign when someone may not understand that sign.

Communication can be difficult when people do not understand the same language. Sign languages are full languages with their own rules. Recognizing a few trained signs is not the same as understanding a full signed conversation.

Our aim is not to replace interpreters or claim that a computer can understand every ISL sentence. We are building a prototype that recognizes a limited set of signs and displays the predicted label as text. The website also has extra speech and translation features.

We will explain how it works, show the prototype and talk about its limits and future improvements.

Show: the homepage and project title.

Handover: I have explained why we chose this problem. Shyamantika will now explain the system and how the recognition steps work.

Simple notes for Barakha

Computer vision means software that looks for useful information in images or video. A landmark is a key point on a hand, such as a fingertip or joint. A feature is a number that describes the hand.

If asked why this project matters, say it is a learning prototype and a starting point for testing a possible accessibility tool.

Do not say it recognizes every ISL sentence, that it is fully offline, or that it replaces an interpreter. Do not guess an accuracy percentage.

Speaker 2 - Shyamantika Pegu

Role: System Design, Technology and Workflow

Suggested time: about 2 minutes

Say:

Thank you, Barakha. I will explain how SAHAYAK works.

First, the user opens the website and allows camera access. The browser uses MediaPipe Hands to find key points on the visible hands. The website turns those points into numbers and sends the numbers to the Python Flask backend. The trained model predicts a label, and the website shows that result.

Each hand has 21 key points. Each point has x, y and z coordinates. For two hands, that gives 2 x 21 x 3 = 126 values for a static pose.

Some signs use movement over time. For those, each frame has 132 values, including hand points and wrist positions. The system arranges the movement into 10 time steps, so the sequence model receives 1,320 values.

The website handles the camera preview and drawing of hand points. The backend runs the model prediction. The current backend uses Python, Flask and scikit-learn-compatible model files saved with Joblib. The frontend is hosted on Vercel, and the backend is hosted on Render.

The website can also have translation or speech features. Those features are separate from recognizing the sign and may need internet access.

Show: a simple diagram or the homepage. If appropriate, point to Live Studio, but leave the main live demo for Himanxhu.

Handover: That is the path from the camera to the predicted label. Himanxhu will now demonstrate the website and explain how to test it.

Simple notes for Shyamantika

Static model: uses one pose input with 126 values.

Continuous model: uses a movement represented by 1,320 values.

A label file names possible classes but does not teach the model a new sign.

Do not claim the project uses TensorFlow or a CNN unless the actual code and model have been checked.

A confidence score is not a guarantee that the result is correct.

Speaker 3 - Himanxhu V Ghimire

Role: Live Demo, Testing and Conclusion

Suggested time: about 2 minutes plus demo

Say:

Thank you. I will show the current SAHAYAK prototype.

I am opening Live Studio. The browser asks for camera permission. When the hand detector finds the hand, the website can draw key points over the camera preview. The program turns those points into numbers and sends them to the backend. The model then tries to predict one of its trained labels.

I will show a sign that appears in the current model's label list. We should look at three things: whether the hand is detected, whether a label appears, and whether the label matches the sign I made. One successful attempt is a demonstration, not proof of overall accuracy.

The project has limits. The model knows only a limited set of signs. Lighting, camera angle, hidden hands, movement speed and similar-looking signs can cause mistakes. Full sign language also uses face movement, body position, grammar and context, which this prototype does not fully understand.

Future work includes more varied labelled examples, fair testing, better moving-sign recognition, clearer privacy information and feedback from people who use ISL.

Thank you for listening. We welcome your questions.

Show: Live Studio, camera permission, hand points, one supported sign and the predicted label. If possible, try an older sign too.

Demo checklist

Before the judges arrive:

- Open the homepage and Live Studio on the device you will use.

- Check internet and camera permission.
- Check backend health and status.
- Confirm active model labels and select a sign from that list.
- Test one or two signs.
- Use good lighting and keep the hands fully inside the camera view.
- Keep a backup diagram or screenshot ready.

During the demo:

1. Tell the judges which sign you will try.
2. Keep the hand visible and make the sign naturally.
3. Wait for the result.
4. Do not hide a wrong result or pretend a failed prediction worked.
5. Explain that translation or speech is separate from recognition.

Backup plan if the live demo fails

Say:

"The live result depends on camera detection and the hosted backend. We have a prototype, and this shows why reliability testing matters. I can explain the steps and show the project interface while we check the connection."

Then use the homepage or a workflow diagram. Do not claim the model worked if there was no result.

Judge questions and simple answers

Q1. What is SAHAYAK?

A: A website prototype that uses hand points and a trained model to recognize a limited set of sign labels.

Q2. What problem does it explore?

A: It explores whether technology can help show the name of a sign in situations where someone may not understand it.

Q3. Is it a complete ISL translator?

A: No. It recognizes a limited set of trained labels. Full sign-language translation is a much bigger task.

Q4. What is MediaPipe Hands?

A: Software that finds key points on hands in camera frames. In this project, it runs in the browser.

Q5. What is a landmark?

A: A key point on a hand, such as a finger joint or fingertip.

Q6. Why use machine learning?

A: It lets the program learn patterns from labelled examples and make a guess for a new example.

Q7. Why 126 values?

A: Two hands x 21 points per hand x three coordinates per point equals 126 numbers.

Q8. Why 1,320 values?

A: Each movement frame has 132 values and the sequence is arranged into 10 time steps.

Q9. Which technology runs the backend?

A: Python and Flask provide the API. The model uses scikit-learn-compatible files saved with Joblib.

Q10. Does it recognize every ISL sign?

A: No. It recognizes only the labels in its active trained model.

Q11. Does it work fully offline?

A: No. Live backend prediction and some translation or speech features need internet.

Q12. How do you know its accuracy?

A: Test with labelled examples not used during training and save the results. Do not quote a number without evidence.

Q13. How will you add a sign?

A: Collect labelled examples, check them, train a candidate model, test it on fresh examples, promote it only if acceptable, then deploy and test again.

Q14. Is adding a name to a label file enough?

A: No. The model needs training examples for the new sign.

Q15. What happens when a prediction is wrong?

A: Record the mistake, check conditions and improve the data or model. Do not hide the result.

Q16. Can it replace an interpreter?

A: No. It is a limited prototype and is not guaranteed to be correct.

Q17. Why is different training data important?

A: It helps the model face real changes between people, cameras and recording conditions.

Q18. How do you protect privacy?

A: Ask permission before using the camera, explain what is sent or stored, and get permission before saving training samples.

Q19. What will you improve next?

A: Collect more balanced examples, test more people and settings, publish clear results and improve weak signs.

Q20. Why is this a science project?

A: It has a clear problem, a method that can be tested, results to check, limits to explain and a plan for improvement.

Closing reminder

Be clear, use simple words and be honest. If a number has not been measured, say so. If a feature fails during the demo, explain the problem and the intended steps. Each person should speak about their own role and give credit for work done by the whole team.